New chat
Chats
Projects
Artifacts
Code

Starred
Test-Kenta
MVMNT-Zul
MVMNT-Dipesh

Recents
NFT transfer restrictions via whi
SmartAccount initialization chai...
Web3auth signer chain configur...
Convert Solidity bridge payload ...
Cross-chain payment amount di...
Cross chain architecture test su...
MVMNTプロジェクトのヘルプセ...
Fixing stack too deep compiler e...
Video hosting for fast loading la...
Solidity fusion contract validatio...
Multi-zone Next.js architecture ...
Next.js architecture for scalable ...
Force distribution on launch close
Migrating Next.js deployment to...
Flaunch contracts decimal handl...
Storing components in Next.js a...

MVMNT
Max plan

using a whitelist / RBAC mechanism in the NFT contract to restrict transfers so NFTs can only be traded on the MVMNT marketplace and not external platforms, accepting some gas/storage compromise to maintain control.
If you want, I can:
* Suggest the best production-grade pattern

Deciphered marketplace restriction intent and formulated production-grade pattern.

I'd be happy to suggest the best production-grade pattern for restricting NFT transfers to your MVMNT marketplace only. Here's a robust approach:

Recommended Pattern: Operator Whitelist with Role-Based Control

solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts-upgradeable/token/ERC721/ERC721Upgradeable.sol";
import "@openzeppelin/contracts-upgradeable/access/AccessControlUpgradeable.sol";
import "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";

abstract contract MarketplaceRestrictedNFT is
    Initializable,
    ERC721Upgradeable,
    AccessControlUpgradeable
{
    bytes32 public constant MARKETPLACE_ROLE = keccak256("MARKETPLACE_ROLE");
    bytes32 public constant WHITELIST_ADMIN_ROLE = keccak256("WHITELIST_ADMIN_ROLE")

    // Mapping of whitelisted operators (your marketplace contracts)
    mapping(address => bool) private _whitelistedOperators;

    // Toggle for restriction (useful for emergency or future flexibility)
    bool public transferRestrictionEnabled;

    event OperatorWhitelisted(address indexed operator, bool status);
    event TransferRestrictionToggled(bool enabled);

    function __MarketplaceRestrictedNFT_init() internal onlyInitializing {
        transferRestrictionEnabled = true;
    }

    // Override the transfer hook - this catches ALL transfer methods
    function _update(
        address to,
        uint256 tokenId,
        address auth
    ) internal virtual override returns (address) {
        address from = _ownerOf(tokenId);

        // Allow minting (from == address(0)) and burning (to == address(0))
        // For transfers, validate the caller
        if (from != address(0) && to != address(0) && transferRestrictionEnabled) {
            _validateTransfer(auth);
        }

        return super._update(to, tokenId, auth);
    }

    function _validateTransfer(address operator) internal view {
        // Allow if caller is the token owner (direct transfer)
        // OR if caller is a whitelisted marketplace
        require(
            _whitelistedOperators[operator] ||
            hasRole(MARKETPLACE_ROLE, operator),
            "Transfer: operator not whitelisted"
        );
    }

    // Admin functions
    function setOperatorWhitelist(
        address operator,
        bool status
    ) external onlyRole(WHITELIST_ADMIN_ROLE) {
        _whitelistedOperators[operator] = status;
        emit OperatorWhitelisted(operator, status);
    }

    function batchWhitelistOperators(
        address[] calldata operators,
        bool status
    ) external onlyRole(WHITELIST_ADMIN_ROLE) {
        for (uint256 i = 0; i < operators.length; i++) {
            _whitelistedOperators[operators[i]] = status;
            emit OperatorWhitelisted(operators[i], status);
        }
    }

    function toggleTransferRestriction(
        bool enabled
    ) external onlyRole(DEFAULT_ADMIN_ROLE) {
        transferRestrictionEnabled = enabled;
        emit TransferRestrictionToggled(enabled);
    }

    function isOperatorWhitelisted(address operator) external view returns (bool) {
        return _whitelistedOperators[operator] || hasRole(MARKETPLACE_ROLE, operator
    }

    // Block setApprovalForAll for non-whitelisted operators
    function setApprovalForAll(
        address operator,
        bool approved
    ) public virtual override {
        if (approved && transferRestrictionEnabled) {
            require(
                _whitelistedOperators[operator] || hasRole(MARKETPLACE_ROLE, operato
                "Approval: operator not whitelisted"
            );
        }
        super.setApprovalForAll(operator, approved);
    }

    // Block approve for non-whitelisted operators
    function approve(
        address to,
```

```
        uint256 tokenId
    ) public virtual override {
        if (to != address(0) && transferRestrictionEnabled) {
            require(
                _whitelistedOperators[to] || hasRole(MARKETPLACE_ROLE, to),
                "Approval: operator not whitelisted"
            );
        }
        super.approve(to, tokenId);
    }
}
```

Usage in Your NFT Contract

```solidity
contract MVMNTDealNFT is MarketplaceRestrictedNFT, PausableUpgradeable {

    function initialize(
        string memory name,
        string memory symbol,
        address marketplace,
        address admin
    ) public initializer {
        __ERC721_init(name, symbol);
        __AccessControl_init();
        __MarketplaceRestrictedNFT_init();

        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(WHITELIST_ADMIN_ROLE, admin);
        _grantRole(MARKETPLACE_ROLE, marketplace);

        // Also add to mapping for gas efficiency on checks
        _whitelistedOperators[marketplace] = true;
    }

    // Your minting logic for auctions, raffles, buynow
    function mintDealNFT(
        address winner,
        uint256 tokenId,
        DealType dealType
    ) external onlyRole(MINTER_ROLE) {
        _safeMint(winner, tokenId);
    }
}
```

Key Benefits of This Pattern

Security: Dual-layer protection via mapping lookup (gas-efficient) and AccessControl roles (auditable, standard).

Flexibility: The `transferRestrictionEnabled` toggle lets you disable restrictions if you ever want to allow open trading or need emergency bypass.

Gas optimization: Mapping checks are cheaper than role checks, so we check mapping first. The role serves as backup and provides AccessControl's event logging.

OpenSea/external marketplace blocking: By restricting `approve` and `setApprovalForAll`, users can't list on OpenSea or other platforms since those marketplaces need approval to transfer.

Future-proofing: You can whitelist additional marketplace contracts (v2, auction-specific, etc.) without upgrading the NFT contract.

Want me to also show the corresponding marketplace contract integration or the subgraph schema for tracking these restricted transfers?

whats the better approach?

Weighed multiple NFT restriction strategies for marketplace exclusivity.

Great question. Let me break down the main approaches and give you a clear recommendation:

Approach Comparison

Approach	Gas Cost	Security	Flexibility	Complexity
Operator Whitelist (mapping)	Low	High	High	Low
Pure RBAC (AccessControl only)	Medium	High	Medium	Low
Blocklist (block OpenSea, etc.)	Low	Low ❌	Low	Low
EIP-712 Signatures	Medium	High	High	High
Soulbound (non-transferable)	Lowest	Highest	None ❌	Lowest

My Recommendation: Simplified Operator Whitelist

For your use case (auction/raffle/buynow marketplace with crypto payments), go with a **lean whitelist using just a mapping** — skip the dual RBAC layer I showed earlier. Here's why:

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts-upgradeable/token/ERC721/ERC721Upgradeable.sol";
import "@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol";

abstract contract MarketplaceRestricted is ERC721Upgradeable, OwnableUpgradeable {

    mapping(address => bool) public whitelistedOperators;

    event OperatorUpdated(address indexed operator, bool status);

    function _update(
        address to,
        uint256 tokenId,
        address auth
    ) internal virtual override returns (address) {
        address from = _ownerOf(tokenId);

        // Only restrict actual transfers (not mint/burn)
        if (from != address(0) && to != address(0)) {
            require(
```

```
            require(
                whitelistedOperators[msg.sender],
                "Only MVMNT marketplace"
            );
        }

        return super._update(to, tokenId, auth);
    }

    function setApprovalForAll(address operator, bool approved) public virtual overr
        if (approved) {
            require(whitelistedOperators[operator], "Operator not allowed");
        }
        super.setApprovalForAll(operator, approved);
    }

    function approve(address to, uint256 tokenId) public virtual override {
        if (to != address(0)) {
            require(whitelistedOperators[to], "Operator not allowed");
        }
        super.approve(to, tokenId);
    }

    function setOperator(address operator, bool status) external onlyOwner {
        whitelistedOperators[operator] = status;
        emit OperatorUpdated(operator, status);
    }
}
```

Why This is Better for MVMNT

1. Simpler = fewer bugs You're already managing complex systems (ICOs, brand tokens, staking, subgraphs). The NFT restriction should be dead simple.

2. Gas efficient Single mapping lookup (~2100 gas) vs AccessControl's role check (~2600+ gas). Adds up across thousands of marketplace trades.

3. You control the marketplace Since you own both the NFT contract and marketplace, you don't need the flexibility of role-based delegation. `onlyOwner` for admin functions is sufficient.

4. Blocklist won't work New marketplaces pop up constantly. Whitelisting your own contracts is the only reliable approach.

5. Soulbound is too restrictive Your deals need to be tradeable on YOUR marketplace — just not elsewhere.

One Enhancement to Consider

If you want an escape hatch for edge cases (legal, migration, etc.):

solidity

```
bool public restrictionsEnabled = true;

function toggleRestrictions(bool enabled) external onlyOwner {
    restrictionsEnabled = enabled;
}

// Then in _update:
if (from != address(0) && to != address(0) && restrictionsEnabled) {
    require(whitelistedOperators[msg.sender], "Only MVMNT marketplace");
}
```

Bottom line: Go with the simple whitelist mapping + Ownable. It's battle-tested, gas-efficient, and matches your architecture where you control the entire ecosystem.

Want me to show how your marketplace contract should call these NFTs for auction wins / raffle claims?

// SPDX-License-Identifier: MIT n MIT pragma solidity ^0. 8.20; import "erc721a-upgr adeable/contracts/ERC72 1AUpgradeable.sol"; Impo rt "@openzeppelin/cont... PASTED

// SPDX-License-Identifier: MIT n MIT pragma solidity ^0. 8.20; import "@openzeppe elin/contracts-upgradeabl e/access/OwnableUpgrad eable.sol"; import "@op... PASTED

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts-
upgradeable/access/OwnableUpgradeable.sol";
import "@openzeppelin/contracts-
upgradeable/proxy/utils/UUPSUpgradeable.sol";
import "@openzeppelin/contracts-
```

Show more

Architected whitelist system restricting NFT transfers to authorized contracts and users. 17s

Looking at your HeroesX contracts, I'll add the whitelist restriction to `HeroesXCore` since that's where all transfers originate. Here's the approach:

Add to HeroesXCore

solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "erc721a-upgradeable/contracts/ERC721AUpgradeable.sol";
import "@openzeppelin/contracts-upgradeable/access/AccessControlUpgradeable.sol";
import "@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol";
import "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";
import "@openzeppelin/contracts/utils/Strings.sol";

contract HeroesXCore is Initializable, ERC721AUpgradeable, AccessControlUpgradeable,
    using Strings for uint256;

    bytes32 public constant MANAGER_ROLE = keccak256("MANAGER_ROLE");
    bytes32 public constant WHITELIST_ADMIN_ROLE = keccak256("WHITELIST_ADMIN_ROLE")

    uint8 public constant FORM_BASE = 0;
    uint8 public constant FORM_HERO = 1;
```

```
    struct TokenData {
        uint8 characterId;
        uint8 form;
    }

    mapping(uint256 => TokenData) public traits;
    mapping(uint8 => mapping(uint8 => uint256[])) private contractTokensByTraits;
    mapping(uint256 => uint256) private tokenIndexInTraitsArray;

    string public baseURI;

    // ============ Whitelist State ============

    /// @notice Whitelisted operators (your contracts: Sales, Fusion, etc.)
    mapping(address => bool) public whitelistedOperators;

    /// @notice Whitelisted users (platform users who can hold/trade NFTs)
    mapping(address => bool) public whitelistedUsers;

    /// @notice Toggle to enable/disable transfer restrictions
    bool public transferRestrictionEnabled;

    // ============ Events ============

    event Minted(address indexed to, uint256 indexed tokenId, uint8 characterId, uin
    event Burned(uint256 indexed tokenId);
    event ContractTransfer(address indexed from, address indexed to, uint256 indexed

    event OperatorWhitelistUpdated(address indexed operator, bool status);
    event UserWhitelistUpdated(address indexed user, bool status);
    event TransferRestrictionToggled(bool enabled);

    // ============ Errors ============

    error OperatorNotWhitelisted(address operator);
    error RecipientNotWhitelisted(address recipient);
    error ApprovalNotAllowed(address operator);

    function initialize(string calldata name_, string calldata symbol_, string calld
        external
        initializerERC721A
        initializer
    {
        __ERC721A_init(name_, symbol_);
        __AccessControl_init();
        __UUPSUpgradeable_init();

        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(WHITELIST_ADMIN_ROLE, msg.sender);

        baseURI = baseURI_;
        transferRestrictionEnabled = true;

        // Whitelist this contract itself (for internal transfers)
        whitelistedOperators[address(this)] = true;
        whitelistedUsers[address(this)] = true;
    }

    // ============ Whitelist Management ============

    /// @notice Set operator whitelist status (for contracts like Sales, Fusion)
    function setOperatorWhitelist(address operator, bool status) external onlyRole(W
        whitelistedOperators[operator] = status;
        emit OperatorWhitelistUpdated(operator, status);
    }

    /// @notice Batch set operator whitelist
    function batchSetOperatorWhitelist(address[] calldata operators, bool status)
        external
        onlyRole(WHITELIST_ADMIN_ROLE)
    {
        for (uint256 i = 0; i < operators.length; i++) {
            whitelistedOperators[operators[i]] = status;
            emit OperatorWhitelistUpdated(operators[i], status);
        }
    }

    /// @notice Set user whitelist status (for platform users)
    function setUserWhitelist(address user, bool status) external onlyRole(WHITELIST
        whitelistedUsers[user] = status;
        emit UserWhitelistUpdated(user, status);
    }

    /// @notice Batch set user whitelist (for onboarding multiple users)
    function batchSetUserWhitelist(address[] calldata users, bool status)
        external
        onlyRole(WHITELIST_ADMIN_ROLE)
    {
        for (uint256 i = 0; i < users.length; i++) {
            whitelistedUsers[users[i]] = status;
            emit UserWhitelistUpdated(users[i], status);
        }
    }

    /// @notice Toggle transfer restrictions (emergency escape hatch)
    function setTransferRestriction(bool enabled) external onlyRole(DEFAULT_ADMIN_RO
        transferRestrictionEnabled = enabled;
        emit TransferRestrictionToggled(enabled);
    }

    /// @notice Convenience function to setup contracts during deployment
    function setupContracts(address salesContract, address fusionContract) external
        _grantRole(MANAGER_ROLE, salesContract);
        _grantRole(MANAGER_ROLE, fusionContract);

        // Also whitelist as operators
        whitelistedOperators[salesContract] = true;
        whitelistedOperators[fusionContract] = true;

        emit OperatorWhitelistUpdated(salesContract, true);
        emit OperatorWhitelistUpdated(fusionContract, true);
    }

    // ============ Transfer Restriction Overrides ============

    /// @notice Override setApprovalForAll to restrict to whitelisted operators only
    function setApprovalForAll(address operator, bool approved) public override {
        if (approved && transferRestrictionEnabled) {
            if (!whitelistedOperators[operator]) {
                revert ApprovalNotAllowed(operator);
            }
        }
        super.setApprovalForAll(operator, approved);
    }
```

```
    /// @notice Override approve to restrict to whitelisted operators only
    function approve(address to, uint256 tokenId) public payable override {
        if (to != address(0) && transferRestrictionEnabled) {
            if (!whitelistedOperators[to]) {
                revert ApprovalNotAllowed(to);
            }
        }
        super.approve(to, tokenId);
    }

    /// @notice Hook that validates transfers
    function _beforeTokenTransfers(
        address from,
        address to,
        uint256 startTokenId,
        uint256 quantity
    ) internal override {
        super._beforeTokenTransfers(from, to, startTokenId, quantity);

        // Existing traits mapping logic
        for (uint256 i = 0; i < quantity; i++) {
            uint256 tokenId = startTokenId + i;

            if (from == address(this)) {
                _removeTokenFromTraitsMapping(tokenId);
            }

            if (to == address(this) && from != address(0)) {
                TokenData memory t = traits[tokenId];
                _addTokenToTraitsMapping(tokenId, t.characterId, t.form);
            }
        }

        // === Transfer Restriction Logic ===
        if (!transferRestrictionEnabled) return;

        // Allow minting (from == address(0))
        if (from == address(0)) {
            // Validate recipient is whitelisted user or operator
            if (!_isWhitelisted(to)) {
                revert RecipientNotWhitelisted(to);
            }
            return;
        }

        // Allow burning (to == address(0))
        if (to == address(0)) return;

        // For transfers: validate operator AND recipient
        // msg.sender is the operator (could be user directly or approved contract)
        if (!whitelistedOperators[msg.sender]) {
            revert OperatorNotWhitelisted(msg.sender);
        }

        // Validate recipient
        if (!_isWhitelisted(to)) {
            revert RecipientNotWhitelisted(to);
        }
    }

    /// @notice Check if address is whitelisted (either as operator or user)
    function _isWhitelisted(address addr) internal view returns (bool) {
        return whitelistedOperators[addr] || whitelistedUsers[addr];
    }

    /// @notice Public view to check if address can receive NFTs
    function isWhitelisted(address addr) external view returns (bool) {
        return _isWhitelisted(addr);
    }

    /// @notice Check if operator can facilitate transfers
    function isOperatorWhitelisted(address operator) external view returns (bool) {
        return whitelistedOperators[operator];
    }

    /// @notice Check if user can hold NFTs
    function isUserWhitelisted(address user) external view returns (bool) {
        return whitelistedUsers[user];
    }

    // ============ Rest of existing functions unchanged ============

    function _authorizeUpgrade(address newImplementation) internal override onlyRole

    function setBaseURI(string calldata newBaseURI) external onlyRole(DEFAULT_ADMIN_
        baseURI = newBaseURI;
    }

    function grantManager(address manager) external onlyRole(DEFAULT_ADMIN_ROLE) {
        _grantRole(MANAGER_ROLE, manager);
    }

    function treasuryMint(address to, uint8 characterId, uint8 form, uint256 quantit
        external
        onlyRole(DEFAULT_ADMIN_ROLE)
    {
        uint256 startId = _nextTokenId();
        _mint(to, quantity);

        for (uint256 i = 0; i < quantity; i++) {
            uint256 tokenId = startId + i;
            traits[tokenId] = TokenData(characterId, form);
            if (to == address(this)) {
                _addTokenToTraitsMapping(tokenId, characterId, form);
            }
            emit Minted(to, tokenId, characterId, form);
        }
    }

    function internalMint(address to, uint8 characterId, uint8 form)
        external
        onlyRole(MANAGER_ROLE)
        returns (uint256 tokenId)
    {
        tokenId = _nextTokenId();
        _mint(to, 1);

        traits[tokenId] = TokenData(characterId, form);
        if (to == address(this)) {
            _addTokenToTraitsMapping(tokenId, characterId, form);
        }
```

```
        emit Minted(to, tokenId, characterId, form);
    }

    function internalBatchMint(address to, uint8 characterId, uint8 form, uint256 qu
        external
        onlyRole(MANAGER_ROLE)
    {
        uint256 startId = _nextTokenId();
        _mint(to, quantity);

        for (uint256 i = 0; i < quantity; i++) {
            uint256 tokenId = startId + i;
            traits[tokenId] = TokenData(characterId, form);
            if (to == address(this)) {
                _addTokenToTraitsMapping(tokenId, characterId, form);
            }
            emit Minted(to, tokenId, characterId, form);
        }
    }

    function burn(uint256 tokenId) external onlyRole(MANAGER_ROLE) {
        if (ownerOf(tokenId) == address(this)) {
            _removeTokenFromTraitsMapping(tokenId);
        }
        _burn(tokenId);
        delete traits[tokenId];
        emit Burned(tokenId);
    }

    function transferFromContract(address to, uint256 tokenId) external {
        require(hasRole(DEFAULT_ADMIN_ROLE, msg.sender) || hasRole(MANAGER_ROLE, msg
        require(ownerOf(tokenId) == address(this), "Token not owned by contract");
        require(to != address(0), "Cannot transfer to zero address");
        require(to != msg.sender, "Cannot transfer to yourself");

        safeTransferFrom(address(this), to, tokenId, "");
        emit ContractTransfer(address(this), to, tokenId);
    }

    function batchTransferFromContract(address to, uint256[] calldata tokenIds) exte
        require(hasRole(DEFAULT_ADMIN_ROLE, msg.sender) || hasRole(MANAGER_ROLE, msg
        require(to != address(0), "Cannot transfer to zero address");
        require(to != msg.sender, "Cannot transfer to yourself");

        for (uint256 i = 0; i < tokenIds.length; i++) {
            uint256 tokenId = tokenIds[i];
            require(ownerOf(tokenId) == address(this), "Token not owned by contract"
            safeTransferFrom(address(this), to, tokenId, "");
            emit ContractTransfer(address(this), to, tokenId);
        }
    }

    function findTokenByTraits(uint8 characterId, uint8 form, uint256 index) externa
        uint256[] storage tokenArray = contractTokensByTraits[characterId][form];
        if (index >= tokenArray.length) {
            return 0;
        }
        tokenId = tokenArray[index];
        if (!_exists(tokenId) || ownerOf(tokenId) != address(this)) {
            return 0;
        }
        return tokenId;
    }

    function getContractTokenCountByTraits(uint8 characterId, uint8 form) external v
        return contractTokensByTraits[characterId][form].length;
    }

    function transferFromContractByTraits(address to, uint8 characterId, uint8 form)
        external
        returns (uint256 tokenId)
    {
        require(hasRole(DEFAULT_ADMIN_ROLE, msg.sender) || hasRole(MANAGER_ROLE, msg
        require(to != address(0), "Cannot transfer to zero address");
        require(to != msg.sender, "Cannot transfer to yourself");

        uint256[] storage tokenArray = contractTokensByTraits[characterId][form];
        require(tokenArray.length > 0, "No matching token found");

        tokenId = tokenArray[0];
        require(ownerOf(tokenId) == address(this), "Token not owned by contract");

        safeTransferFrom(address(this), to, tokenId, "");
        emit ContractTransfer(address(this), to, tokenId);
    }

    function batchTransferFromContractByTraits(address to, uint8 characterId, uint8
        external
        returns (uint256 transferredCount)
    {
        require(hasRole(DEFAULT_ADMIN_ROLE, msg.sender) || hasRole(MANAGER_ROLE, msg
        require(to != address(0), "Cannot transfer to zero address");
        require(to != msg.sender, "Cannot transfer to yourself");
        require(quantity > 0, "Quantity must be greater than 0");

        uint256[] storage tokenArray = contractTokensByTraits[characterId][form];
        uint256 availableCount = tokenArray.length;
        require(availableCount > 0, "No matching tokens found");

        uint256 transferCount = quantity < availableCount ? quantity : availableCoun

        for (uint256 i = 0; i < transferCount; i++) {
            uint256 tokenId = tokenArray[0];
            require(ownerOf(tokenId) == address(this), "Token not owned by contract"

            safeTransferFrom(address(this), to, tokenId, "");
            emit ContractTransfer(address(this), to, tokenId);

            transferredCount++;
        }
    }

    function getTraits(uint256 tokenId) external view returns (uint8 characterId, ui
        require(_exists(tokenId), "Nonexistent token");
        TokenData memory t = traits[tokenId];
        return (t.characterId, t.form);
    }

    function tokenURI(uint256 tokenId) public view override returns (string memory)
        require(_exists(tokenId), "Nonexistent token");
        require(bytes(baseURI).length > 0, "Base URI not set");
        return string(abi.encodePacked(baseURI, tokenId.toString()));
    }
```

```
    function _addTokenToTraitsMapping(uint256 tokenId, uint8 characterId, uint8 form
        uint256[] storage tokenArray = contractTokensByTraits[characterId][form];
        uint256 index = tokenArray.length;
        tokenArray.push(tokenId);
        tokenIndexInTraitsArray[tokenId] = index;
    }

    function _removeTokenFromTraitsMapping(uint256 tokenId) private {
        TokenData memory t = traits[tokenId];
        uint256[] storage tokenArray = contractTokensByTraits[t.characterId][t.form]

        if (tokenArray.length == 0) return;

        uint256 index = tokenIndexInTraitsArray[tokenId];
        uint256 lastIndex = tokenArray.length - 1;

        if (index != lastIndex) {
            uint256 lastTokenId = tokenArray[lastIndex];
            tokenArray[index] = lastTokenId;
            tokenIndexInTraitsArray[lastTokenId] = index;
        }

        tokenArray.pop();
        delete tokenIndexInTraitsArray[tokenId];
    }

    function supportsInterface(bytes4 interfaceId)
        public
        view
        override(ERC721AUpgradeable, AccessControlUpgradeable)
        returns (bool)
    {
        return super.supportsInterface(interfaceId);
    }
}
```

Deployment / Setup Flow

```solidity
// 1. Deploy HeroesXCore
heroesXCore.initialize("HeroesX", "HRX", "https://api.heroesx.com/");

// 2. Deploy Sales and Fusion contracts
heroesXSales.initialize(...);
heroesXFusion.initialize(address(heroesXCore));

// 3. Setup contracts (grants MANAGER_ROLE + whitelists operators)
heroesXCore.setupContracts(address(heroesXSales), address(heroesXFusion));

// 4. Whitelist platform users (can be done via backend)
heroesXCore.batchSetUserWhitelist([user1, user2, user3], true);
```

Backend Integration (Nest.js)

You'll need a service to whitelist users when they register on your platform:

```typescript
// whitelist.service.ts
@Injectable()
export class WhitelistService {
  constructor(
    private readonly ethersService: EthersService,
  ) {}

  async whitelistUser(userAddress: string): Promise<string> {
    const heroesXCore = this.ethersService.getHeroesXCore();
    const tx = await heroesXCore.setUserWhitelist(userAddress, true);
    return tx.hash;
  }

  async batchWhitelistUsers(addresses: string[]): Promise<string> {
    const heroesXCore = this.ethersService.getHeroesXCore();
    const tx = await heroesXCore.batchSetUserWhitelist(addresses, true);
    return tx.hash;
  }

  async removeUserFromWhitelist(userAddress: string): Promise<string> {
    const heroesXCore = this.ethersService.getHeroesXCore();
    const tx = await heroesXCore.setUserWhitelist(userAddress, false);
    return tx.hash;
  }
}
```

Key Points

Feature	Behavior
OpenSea/external marketplaces	Blocked (can't approve non-whitelisted operators)
Your Sales contract	Whitelisted operator, can transfer
Your Fusion contract	Whitelisted operator, can burn/mint
Platform users	Must be whitelisted to receive NFTs
Emergency toggle	`setTransferRestriction(false)` disables all restrictions

This gives you full control over where NFTs can go while keeping gas costs minimal (single mapping lookup ~2100 gas).

so only hexa core needs an update right?

Validated that only HeroesXCore requires modification for whitelist functionality.

Yes, exactly. Only `HeroesXCore` needs the update.

Here's why:

Contract	Change Needed?	Reason

HeroesXCore	✅ Yes	All transfers flow through `_beforeTokenTransfers` hook here
HeroesXSales	❌ No	Calls `heroesXCore.transferFrom()` — gets validated by Core
HeroesXFusion	❌ No	Calls `heroesXCore.burn()` / `internalMint()` — gets validated by Core

Transfer Flow

```
User tries to list on OpenSea
    └── Calls setApprovalForAll(opensea, true)
        └── HeroesXCore.setApprovalForAll() ❌ BLOCKED (opensea not whitelisted)

User buys via HeroesXSales
    └── Sales calls heroesXCore.transferFrom(seller, buyer, tokenId)
        └── _beforeTokenTransfers()
            ├── Is msg.sender (Sales) whitelisted operator? ✅
            └── Is buyer whitelisted user? ✅
                └── Transfer succeeds ✅

User fuses via HeroesXFusion
    └── Fusion calls heroesXCore.burn() + internalMint()
        └── _beforeTokenTransfers()
            ├── Is msg.sender (Fusion) whitelisted operator? ✅
            └── Is user whitelisted? ✅
                └── Burn/mint succeeds ✅
```

The core contract acts as the single gatekeeper — no changes needed to your Sales or Fusion contracts.

all good?

Identified critical duplicate function error in contract code.

Almost! There's one issue — **duplicate `setupContracts` function.** You have it defined twice:

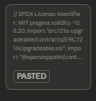
```solidity
// First one (line ~95) ✅ Correct - has whitelist logic
function setupContracts(address salesContract, address fusionContract) external only
    _grantRole(MANAGER_ROLE, salesContract);
    _grantRole(MANAGER_ROLE, fusionContract);

    // Also whitelist as operators
    whitelistedOperators[salesContract] = true;
    whitelistedOperators[fusionContract] = true;

    emit OperatorWhitelistUpdated(salesContract, true);
    emit OperatorWhitelistUpdated(fusionContract, true);
}

// Second one (line ~136) ❌ Delete this - no whitelist logic
function setupContracts(address salesContract, address fusionContract) external only
    _grantRole(MANAGER_ROLE, salesContract);
    _grantRole(MANAGER_ROLE, fusionContract);
}
```

Remove the second one (the one without whitelist logic). This will cause a compilation error as-is.

Everything else looks good:

Check	Status
Whitelist state variables	✅
`WHITELIST_ADMIN_ROLE`	✅
`_beforeTokenTransfers` restriction logic	✅
`setApprovalForAll` override	✅
`approve` override	✅
Self-whitelisting in `initialize`	✅
Events & custom errors	✅
View functions (`isWhitelisted`, etc.)	✅

After removing the duplicate, it should compile and work correctly.



Identified duplicate function definitions causing compilation errors.

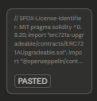
Still have **duplicate `setupContracts`** — same function appears twice (lines ~108 and ~148). Delete one of them:

```solidity
// Line ~108 - KEEP THIS ONE
function setupContracts(address salesContract, address fusionContract) external only
    _grantRole(MANAGER_ROLE, salesContract);
    _grantRole(MANAGER_ROLE, fusionContract);

    whitelistedOperators[salesContract] = true;
    whitelistedOperators[fusionContract] = true;
```

```
    emit OperatorWhitelistUpdated(salesContract, true);
    emit OperatorWhitelistUpdated(fusionContract, true);
}

// Line ~148 - DELETE THIS ONE (exact duplicate)
function setupContracts(address salesContract, address fusionContract) external only
    _grantRole(MANAGER_ROLE, salesContract);
    _grantRole(MANAGER_ROLE, fusionContract);

    whitelistedOperators[salesContract] = true;
    whitelistedOperators[fusionContract] = true;

    emit OperatorWhitelistUpdated(salesContract, true);
    emit OperatorWhitelistUpdated(fusionContract, true);
}
```

This will cause a compilation error: `DeclarationError: Function with same name and parameter types defined twice.`

Remove the second one and you're good to go. ✅



Audited contract architecture and validated whitelist implementation thoroughly.

Looks good now! ✅

Summary of what's in place:

Feature	Status
No duplicate functions	✅
`WHITELIST_ADMIN_ROLE`	✅
`whitelistedOperators` mapping	✅
`whitelistedUsers` mapping	✅
`transferRestrictionEnabled` toggle	✅
Self-whitelist in `initialize()`	✅
`setApprovalForAll` override	✅
`approve` override	✅
`_beforeTokenTransfers` restriction logic	✅
View functions	✅

Transfer restriction flow:

```
Mint (from = 0x0)      → Recipient must be whitelisted ✅
Burn (to = 0x0)        → Always allowed ✅
Transfer               → Operator must be whitelisted + Recipient must be
whitelisted ✅
Approve/ApprovalForAll → Only whitelisted operators ✅
```

Ready to compile and deploy. 🚀

Reply...

Opus 4.5